

Mutation free-energy labels improve low-data nanobody thermal stability prediction

111.2

1
2

[†]These authors contributed equally to this work.

Thermal stability limits how far a nanobody can be engineered, yet experimental melting-temperature (T_m) measurements are scarce. Simulations and physics-based calculations can label far more sequence variants, but they report quantities related to T_m rather than T_m itself. We tested these labels as auxiliary supervision for low-data T_m prediction, with model selection tied only to experimental T_m . On a public benchmark of 57 training, 114 validation, and 396 held-out test sequences, we trained multi-task models in which a T_m predictor shared a protein-language-model representation with one computational source task at a time. The source tasks ranged from mutation free energies and structure-based mutation scores to Rosetta scores for language-model-proposed variants and molecular-dynamics (MD) trajectory summaries. Alchemical free-energy perturbation (FEP) helped most. It lowered held-out T_m mean absolute error from 6.61 to 6.26°C, a consistent paired gain of 0.35°C (90% confidence interval, 0.24–0.46°C). Enlarging the ESM2 encoder did not reproduce this gain, and an MD-derived Q-value did not deliver it at all. Rosetta-scored proposal sets also worked as source labels, but their edge over random variants stayed within noise. The value of a computational label therefore depends on whether its physical meaning carries over to the experimental endpoint, not on how many labels it supplies.

Machine learning can now screen enormous molecular and materials spaces before any wet-lab test, but it is only as good as the property labels it learns from. In materials informatics, this label bottleneck motivated a simulation-to-real (Sim2Real) strategy: train on large computational databases generated by first-principles calculations, quantum chemistry, or molecular dynamics, and transfer the resulting model to a smaller experimental target set (1–3). For polymers and inorganic materials, the transferred error can fall predictably as the computational database grows, which gives a concrete way to judge whether more simulation is worth generating (3). The premise matters because simulation data are not useful by default: model assumptions, finite sampling, and systematic domain gaps separate any calculation from the measurement it stands in for.

Protein engineering has the same bottleneck with an extra complication. Experimental measurements are expensive and sparse, while simulations and physics-based calculations

can label many designed or mutated sequences. But the simulated quantity is often not the experimental target. A protein’s melting temperature (T_m)—the temperature at which half of the molecules are unfolded under a given assay—is one such target. What is tractable to compute is usually something adjacent: how much a mutation shifts stability, a structure-based stability score, or a summary of a molecular-dynamics trajectory. The question is therefore not whether simulation data can be added to training, but which simulated quantities transfer to the experimental phenotype.

We study this question in nanobodies. These single-domain VHH antibodies are appealing scaffolds—small, modular, and robust binders (4, 5)—but their use in expression, storage, and formulation hinges on thermal stability, and measuring or simulating that stability directly is costly (6). Sequence-based predictors built on protein language models (PLMs) are an attractive shortcut: models such as ESM2 learn general-purpose representations from large sequence corpora (7, 8), and several methods now read T_m out of such representations (9–11). High-throughput assays are reshaping the data landscape as well: Tsuboyama et al. used cDNA-display proteolysis to measure folding stability for roughly 776,000 natural and designed domains (12). That scale is remarkable, but it does not yet reach nanobodies. The curated domains span 40–72 residues, whereas VHH domains are considerably longer, and comparable high-throughput ΔG data for them do not exist; a model fine-tuned on the mega-scale set also transferred less well to 177–501-residue proteins than to small domains (13). Large stability resources are powerful, then, but target-specific supervision and careful validation remain essential for nanobody T_m prediction (14–16).

Here we test whether simulation-derived labels can supply that supervision when experimental T_m is scarce. We treat T_m prediction as the target task and each computational label as a source task that shapes a shared sequence representation, with ESM2 as the encoder (8). The source labels span several kinds of information: mutation free energies from alchemical free-energy perturbation (FEP), Rosetta mutation scores, ThermoMPNN stability scores, Rosetta scores on variants proposed by ESM2 or drawn at random, and an MD-derived Q-value that tracks native-contact persistence. To keep the comparison honest, every model setting is chosen on an experimental validation set and every final number comes from the same held-out experimental T_m test set, using paired error comparisons. This prevents a model that excels at a computational task from being mistaken for a better

Tm predictor.

The benefit turns out to be source-dependent. FEP mutation free energies give the clearest improvement over training on Tm alone, whereas the MD-derived Q-value gives none under the same protocol. Rosetta-scored ESM2 proposals give a weaker, exploratory signal that points toward future generator–physics–predictor design loops without demonstrating closed-loop optimization. Together these results carry the materials-informatics Sim2Real argument into a setting where the source and target labels differ in physical meaning: useful simulation data are not merely abundant, they encode a quantity that transfers to the experimental property of interest.

The study tests whether a computational source label can improve experimental nanobody Tm prediction when that label is physically related to Tm but not identical to it. We used a multi-task transfer-learning design: experimental Tm prediction was the target task, and each computational label was a source task that shared part of the network with the target. Throughout, model selection and every reported number were tied to experimental Tm performance. Source-task losses shaped the shared representation during training, but source-task validation errors were never used to pick checkpoints or hyperparameters.

Experimental Tm labels came from the public nanobody benchmark NbBench, which provides a thermo-seq task with predefined splits (15). We deliberately assigned the smallest partition to training and the largest to the held-out test set, giving 57 training, 114 validation, and 396 test sequences. This places the model in the low-data regime that the study targets: scarce experimental Tm labels, not abundant ones, are the condition under which simulation-derived supervision is expected to help. The large test set is a direct benefit of this choice, because it makes paired error comparisons and label-count trends easier to resolve.

Split definitions. The training split fit model parameters. The validation split drove checkpoint selection, early stopping, and hyperparameter choice. The test split stayed untouched until the settings were fixed and was used only for final reporting. The same rule held for Tm-only models and for multi-task transfer models.

Target-label scaling. The target CSV files held an amino-acid sequence column and a Tm label in degrees Celsius. During training, Tm labels were mapped to a scaled regression target: a robust scaling step followed by min–max scaling to [0,1], with the scaler fit only on the 57 training labels. The fitted scaler was then applied to validation labels during training. At evaluation, predictions were inverse-transformed back to degrees Celsius before errors were computed, so the validation and test label distributions never entered the scaling transform.

Experimental-label scaling reference. For this reference, the Tm training split was subsampled to the requested number of labels using the run seed, while the validation and test splits were left intact. These runs give a target-label baseline for judging whether computational source-label scaling moves in the expected direction.

Computational labels were loaded as source tasks, not as additional Tm measurements.

Source-label classes. The source labels were mutation free-energy changes from alchemical free-energy perturbation (FEP), structure-based mutation-effect scores from Rosetta, learned mutation-effect scores from ThermoMPNN, Rosetta scores on ESM2-proposed and random variants, and an MD-derived Q-value summarizing native-contact persistence. All source labels were preprocessed to [0,1]-scaled regression targets and entered the model only through their own source-task heads.

Processed source-table sizes. Mutation-effect sources were organized as two template-specific tables derived from the 1MEL and 4IDL nanobody structures. FEP, Rosetta, and ThermoMPNN each contained 435 and 409 processed rows for the two templates. ESM2-proposed variants and random variants scored by Rosetta each contained 1000 and 1000 processed rows. The MD-derived Q-value source came from a separate panel of simulated single-domain antibody structures drawn from SAbDab, the Structural Antibody Database (17); its processed table contained 1143 analysis-ready rows. Row counts exclude headers.

Sequence-overlap check. We checked exact sequence overlap between the source-label tables and the NbBench splits. The mutation-effect tables and the Rosetta-scored variant tables, which carry explicit sequence columns, had no exact match to any NbBench training, validation, or test sequence. The SAbDab-derived MD table contained a few exact matches, including eight test-set sequences. Because the MD Q-value source did not improve final Tm prediction, this overlap cannot account for the positive FEP result; we report it for transparency.

Source-label sampling. For a mutation-effect experiment with requested count n , up to n rows were sampled independently from each of the two template tables using the run seed, and the two tables were given separate task identifiers. For an MD-derived Q-value experiment with count n , n rows were sampled from the single Q-value table. Sampled rows were then split 80/20 into source-training and source-validation rows with the same seed. The source-validation rows were kept for monitoring and control analyses, but in the reported protocol the validation data passed to the trainer for checkpoint selection and early stopping were filtered down to the experimental Tm validation rows.

The MD source labels were built from nanobody structures selected from a SAbDab-derived summary table (17). We took one heavy-chain nanobody structure per PDB entry

when an IMGT-renumbered coordinate file was available and the chosen chain could be verified in that file. The 400 K simulation panel held 1146 nanobody systems. Requiring a usable trajectory, topology, reference structure, a sequence of at least 50 residues, and at least one selected native contact left 1143 trajectories in the processed table.

Each system was prepared from the selected nanobody chain alone. We used the IMGT-renumbered PDB as the structure source, extracted the heavy chain, dropped non-protein components, left the termini uncapped, and assigned protonation at pH 7.4. Structure cleaning and missing-atom handling used PDBFixer, followed by pdb2pqr/PROPKA for pH-aware protonation and Amber-compatible atom naming when available (18, 19). Systems were solvated in explicit OPC water with 15 Å padding and 0.15 M NaCl, and Amber topologies were built with the ff19SB protein force field and OPC water model (20–22).

Trajectories were run with OpenMM on CUDA devices (23). Each system was equilibrated at 300 K with 1 ns NVT and 1 ns NPT at 1 atm, using a 2-fs timestep without hydrogen-mass repartitioning. The 400 K branch was then started from the 300 K equilibrated state through two restrained NVT relaxation stages—1 ns at 400 K with $C\alpha$ restraints, then a second 1 ns 400 K restrained NVT. Production ran 100 ns of restraint-free 400 K NVT with hydrogen-mass repartitioning (4 amu hydrogen mass), a 4-fs timestep, Langevin middle integration at a 1 ps^{-1} collision rate, PME electrostatics with a 1.0-nm nonbonded cutoff, and coordinate/energy output every 100 ps. The MD source labels use this 400 K production branch.

Q-values were extracted from the 400 K production trajectories with MDTraj (24). For each trajectory, the topology was loaded from the Amber parameter file and the native reference was frame 0 of production. A smooth native-contact fraction (25) was computed from backbone-heavy-atom contacts and averaged over the terminal portion of the trajectory, then min–max scaled to [0,1] before training. The exact contact selection, contact function, averaging window, and column convention are given in Supplementary Methods.

The FEP source labels were mutation free-energy changes for variants of the 1MEL and 4IDL nanobody templates, computed with the same protocol as our related nanobody thermostability study. In brief, alchemical mutation simulations were run with NAMD 3 using GPU acceleration (26). Input systems were prepared in VMD (27) with a customized AlaScan workflow (28), and coordinates were initialized from AlphaFold 3 models of the wild-type sequences (29). Systems used the CHARMM36 protein force field (30), TIP3P water (31), and 150 mM NaCl.

Each transformation used a dual-topology mutation setup with a CHARMM36 hybrid-residue topology, split into 20 equally spaced λ windows from wild type to mutant. Forward and backward transformations were run for 1 ns per direction with a 2-fs timestep and 200 ps of per-window equilibration before data collection, all in the NPT ensemble at 300 K and 1

atm with Langevin temperature control and a Langevin piston barostat. Bonds to hydrogen were constrained with SHAKE (32), and long-range electrostatics used particle mesh Ewald (33). Relative free energies were estimated with the Bennett acceptance ratio (34). The processed tables retain the raw mutation free-energy value, its negated value, and the min–max scaled label used by the network; under this convention, larger processed labels correspond to more favorable mutation free energies.

Rosetta mutation-effect labels were generated with the Rosetta application (35). For each template, the PDB structure was cleaned to the target chain, variant sequences were compared with the wild-type sequence, and the differences were written to Rosetta mutation-list files. The calculation used full-atom Rosetta with the score function (36), 50 iterations, Cartesian minimization enabled, local-only optimization disabled, side-chain-minimization-only disabled, minimum-score aggregation, and no PDB dumping. The same protocol scored the template mutation-effect tables, the ESM2-proposed variants, and the random-variant control.

For the ESM2-proposed source, ESM2 650M sampled 100,000 two-mutation variants from each wild-type template, ranked them by pseudo-log-likelihood, and kept the top 1% for Rosetta scoring (8). The random-variant control used roughly 1000 two-mutation variants with positions and substitutions chosen at random, scored by the same Rosetta workflow.

ThermoMPNN source labels were computed from the same template-specific variant sets with ThermoMPNN, a stability-change predictor trained by transfer learning from larger protein-stability data (37), and converted to the same processed CSV format as the other mutation-effect sources.

All models used an ESM2 protein language-model encoder (8) with lightweight regression heads. Unless stated otherwise, the encoder was the 8M-parameter ESM2 model. Sequences were tokenized with truncation and padding to 160 residues. The first-token encoder representation fed a shared multilayer perceptron with hidden dimensions 256, 128, and 32, ReLU activations, and dropout. A scalar T_m head predicted the scaled target label; in source-transfer models, additional scalar heads predicted the source labels from the same shared representation.

The T_m -only baseline used the encoder, the shared trunk, and the T_m head with experimental T_m labels alone. Source-transfer models kept this target path and added source heads. The main FEP model used separate source heads for the two template-specific mutation-effect tasks, the best design in our source-head comparison. As controls we also tried a shared mutation-effect head, a shared head conditioned on a template embedding, and a shared head with template-specific affine calibration, along with residual and latent source-fusion variants; the main result used the original shared-trunk architecture.

We evaluated two encoder regimes. In the hot-encoder regime, the ESM2 encoder and the regression heads were optimized jointly, with the encoder given a smaller learning rate than the freshly initialized trunk and heads so that the pre-trained representation was updated more gently. In the final FEP setting, the selected encoder learning rate was 3×10^{-5} and the head learning rate was the default 3×10^{-4} .

In the frozen-encoder control, all ESM2 parameters were fixed. The encoder was run once to precompute sequence embeddings for the target and source examples, and training then optimized only the trunk and task heads. This control asks whether the source labels still help when the pretrained representation is used as a fixed feature extractor. Encoder-size controls repeated selected Tm-only and FEP settings with 35M- and 650M-parameter ESM2 encoders.

Each training example was a triple (x_i, y_i, t_i) : sequence x_i , scaled label y_i , and task identifier t_i . The target Tm task had one identifier, and each source task had its own. A mini-batch could mix target and source examples, but every example contributed loss only through the head for its task.

The per-task regression loss was Huber loss with $\delta = 1.0$ on scaled labels. The default multi-task objective used learnable task-uncertainty weights (38): for task k with loss L_k and learned log-scale s_k , the contribution was

$$\frac{L_k}{2 \exp(2s_k)} + s_k,$$

and the total loss summed over the tasks present in the mini-batch. Fixed source-task weights were also evaluated as controls, particularly for the MD-derived labels.

Models were trained with AdamW, cosine learning-rate scheduling, batch size 16, weight decay 0.04, 100 warmup steps, and at most 400 epochs. The default head learning rate was 3×10^{-4} , the default hot-encoder learning rate was 1×10^{-4} , and the default dropout was 0.15. Training used mixed precision on CUDA devices, with checkpoints saved and evaluated once per epoch.

Within each run, the best checkpoint was the one with the lowest mean squared error on the experimental Tm validation examples, and early stopping used the same metric with a patience of 15 epochs. Source-label rows were present in the training set, but they were removed from the validation data passed to the trainer for checkpoint selection and early stopping. This is the implementation-level step that keeps reported checkpoints selected by target-task validation performance rather than by source-task performance.

Hyperparameters were likewise selected on the experimental Tm validation split. Candidate settings varied the encoder learning rate, head learning rate, dropout, loss-weighting mode, source-head design, and selected architecture controls. The search covered encoder learning rates of 3×10^{-5} , 1×10^{-4} , and 3×10^{-4} ; a lower head learning

rate of 1×10^{-4} paired with encoder learning rate 3×10^{-5} ; dropout rates of 0.05, 0.15, and 0.30; and fixed source-task loss weights where relevant. Each candidate was evaluated as a three-seed ensemble on the Tm validation split, and the selected setting for each source class was then evaluated on the held-out test split as a ten-seed ensemble.

Independent run seeds controlled model initialization, target-training subsampling in the target-label scaling experiment, source-label sampling, and source-task train/validation splitting. Final comparisons used seeds 1–10. The label-count curves used fixed source-specific training recipes while varying the number of source labels, rather than retuning a separate model at every label-count point.

Final evaluation used the same 396 held-out Tm test examples for every condition. For each condition, ten independently seeded models produced scaled Tm predictions for every test sequence; these were averaged in scaled-label space and inverse-transformed to degrees Celsius. The primary metric was mean absolute error (MAE) in degrees Celsius. Root mean squared error, coefficient of determination, and Spearman and Pearson correlations were also computed, but MAE anchored the main comparisons because it reads directly as an average temperature error.

Confidence intervals came from nonparametric bootstrap resampling over held-out test proteins. For a single condition, the absolute-error vector from the ten-seed ensemble was re-sampled with replacement 10,000 times using a fixed random seed, and the reported 90% interval is the 5th–95th percentile range of the bootstrapped MAE values.

For paired comparisons, the same bootstrap indices were applied to the absolute-error vectors of the candidate and reference conditions, and the candidate-minus-reference MAE difference was computed for each bootstrap sample. This paired bootstrap preserves the matched-test-example structure of the comparison. Throughout, ΔMAE is candidate minus reference, so negative values mean lower error than the reference. The evaluation code also reports the one-sided tail probability—the fraction of paired bootstrap samples with ΔMAE above zero—but we emphasize effect sizes and 90% confidence intervals.

The pipeline builds the target and source tasks, applies target-label scaling, performs seeded source-label sampling, filters validation data for target-task checkpoint selection, evaluates seed ensembles, and writes bootstrap statistics. Each run emits a machine-readable summary with the command-line arguments, resolved model settings, per-point metrics, bootstrap intervals, and paired comparisons. These summaries feed the figure-generation workflow directly and will be archived with the processed data for reproducibility.

We first built the evaluation framework. The target task was nanobody Tm regression from amino-acid sequence, and computational labels were treated as source prediction tasks, not as stand-ins for experimental Tm. Each model used the pretrained protein sequence model ESM2 to turn a sequence into a learned representation, which a Tm head and a computed-label head shared. The source task could therefore reshape the representation while the final objective stayed anchored to experimental Tm (Fig. 1).

Every candidate setting was selected on an experimental Tm validation set, and the final numbers came only from a held-out experimental Tm test set; improvements were measured as paired differences in absolute error on the same test examples. This guards against a trap specific to simulation-to-experiment learning: a computational task can be learnable and physically sensible yet do nothing for the experimental phenotype.

We next varied the number of experimental and computational labels to see whether more source data meant better Tm prediction. Adding experimental Tm labels helped the Tm-only model, as expected (Fig. 2). The target-label sweep used 10, 20, 30, 40, and 57 experimental Tm training labels; the FEP sweep used 10, 40, 80, 160, and 320 rows per template-specific source table; and the MD Q-value sweep used 10, 40, 80, 160, 320, and 640 rows from the single MD source table. FEP mutation free-energy labels showed a beneficial trend: the largest FEP setting gave the best held-out Tm error among the FEP points. The curve was not strictly monotonic, so we do not read it as a clean power law. The consistent reading is that more mutation free-energy labels help when models are selected and judged through the experimental Tm endpoint.

The MD-derived Q-value behaved differently. This label measures how well native contacts persist during high-temperature MD trajectories. Adding more of these labels did not reliably beat learning from Tm measurements alone, even though they too came from simulation. The sweep thus sets up the central comparison below: label count alone is not enough; what matters is the transferable physical content of the label.

We then focused on two all-atom source labels. Training on experimental Tm alone reached a held-out test MAE of 6.61°C. Adding FEP mutation free-energy labels cut this to 6.26°C, a consistent paired Δ MAE of -0.35°C (90% confidence interval, -0.46 to -0.24°C; Fig. 3); negative paired changes mean lower error than the Tm-only reference. The MD-derived Q-value went the other way, raising the test error to 6.73°C (paired change, +0.12°C; 90% confidence interval, +0.01 to +0.22°C). Both labels came from atomistic simulation, yet only the mutation free energy improved the experimental endpoint.

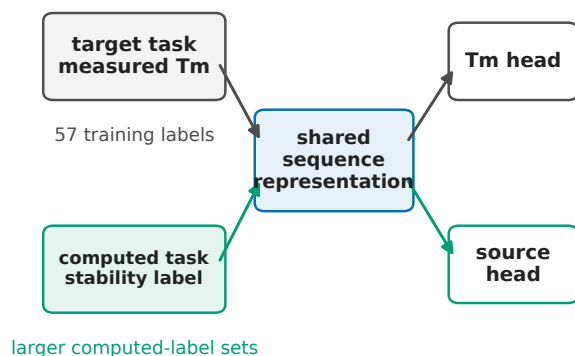
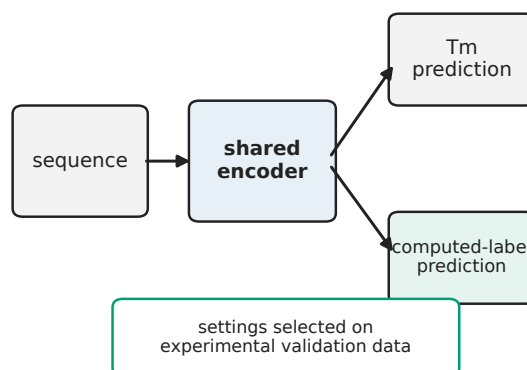
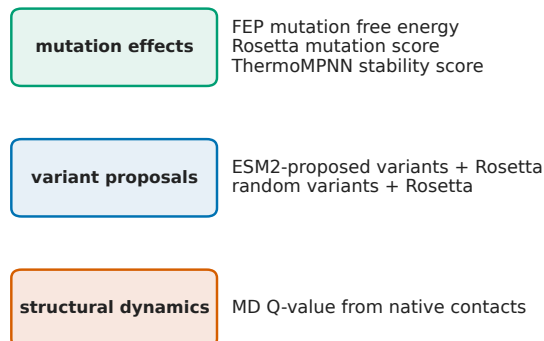
The encoder controls reinforced this split. With a frozen encoder, errors rose overall, but FEP still beat the frozen Tm-only baseline: 7.20°C versus 7.38°C MAE. The gain was smaller than with a hot encoder, and the frozen FEP model stayed worse than the hot-encoder FEP model. The MD Q-value helped in neither regime. FEP labels therefore carry useful information even when the representation is fixed, but adapting the encoder is what extracts the largest target-task gain.

Because a protein language model's size can drive downstream performance, we checked whether a larger encoder alone could reproduce the FEP benefit. It could not. In the Tm-only setting, the 35M and 650M encoders did not beat the 8M encoder on held-out performance. In the FEP setting, the 35M and 650M encoders reached 6.50 and 6.65°C MAE, both worse than the 8M FEP model. FEP labels stayed beneficial within each size comparison, but the best absolute performance came from the smallest encoder paired with FEP labels. Within the tested range, the gain tracked the physical source label, not the scale of the language model (Supplementary Fig. 4).

We next looked at computational labels outside the FEP and MD pipelines. Rosetta scores on ESM2-proposed variants reached 6.45°C MAE (paired change, -0.17°C; 90% confidence interval, -0.31 to -0.03°C), and ThermoMPNN stability scores reached 6.46°C MAE (paired change, -0.15°C; 90% confidence interval, -0.30 to -0.01°C; Fig. 4). Rosetta mutation scores reached 6.53°C MAE (paired change, -0.09°C; 90% confidence interval, -0.20 to +0.02°C), and random variants scored by Rosetta reached 6.51°C MAE (paired change, -0.10°C; 90% confidence interval, -0.22 to +0.03°C). All were weaker than FEP, and the two Rosetta controls had confidence intervals that crossed zero.

The ESM2-proposed variants scored by Rosetta edged out the random variants, with a paired MAE difference of -0.07°C in favor of the ESM2-proposed set. The confidence interval crossed zero, so we read this as exploratory rather than as evidence that the present generator improves variant selection. It does show, though, that language-model proposals can be annotated by physics-based scoring and used as source labels for the Tm predictor—a feasibility point we return to in the Discussion.

Across the labels we tested, the strongest and most consistent improvement came from FEP mutation free energies. Other structure-based labels and proposal-set scores gave weaker gains, and the MD-derived Q-value gave none. The pattern is selective, not additive: a computational label helps only when the information it carries transfers to the experimental Tm endpoint.

A**B****C****D**

Final claims use held-out Tm test errors

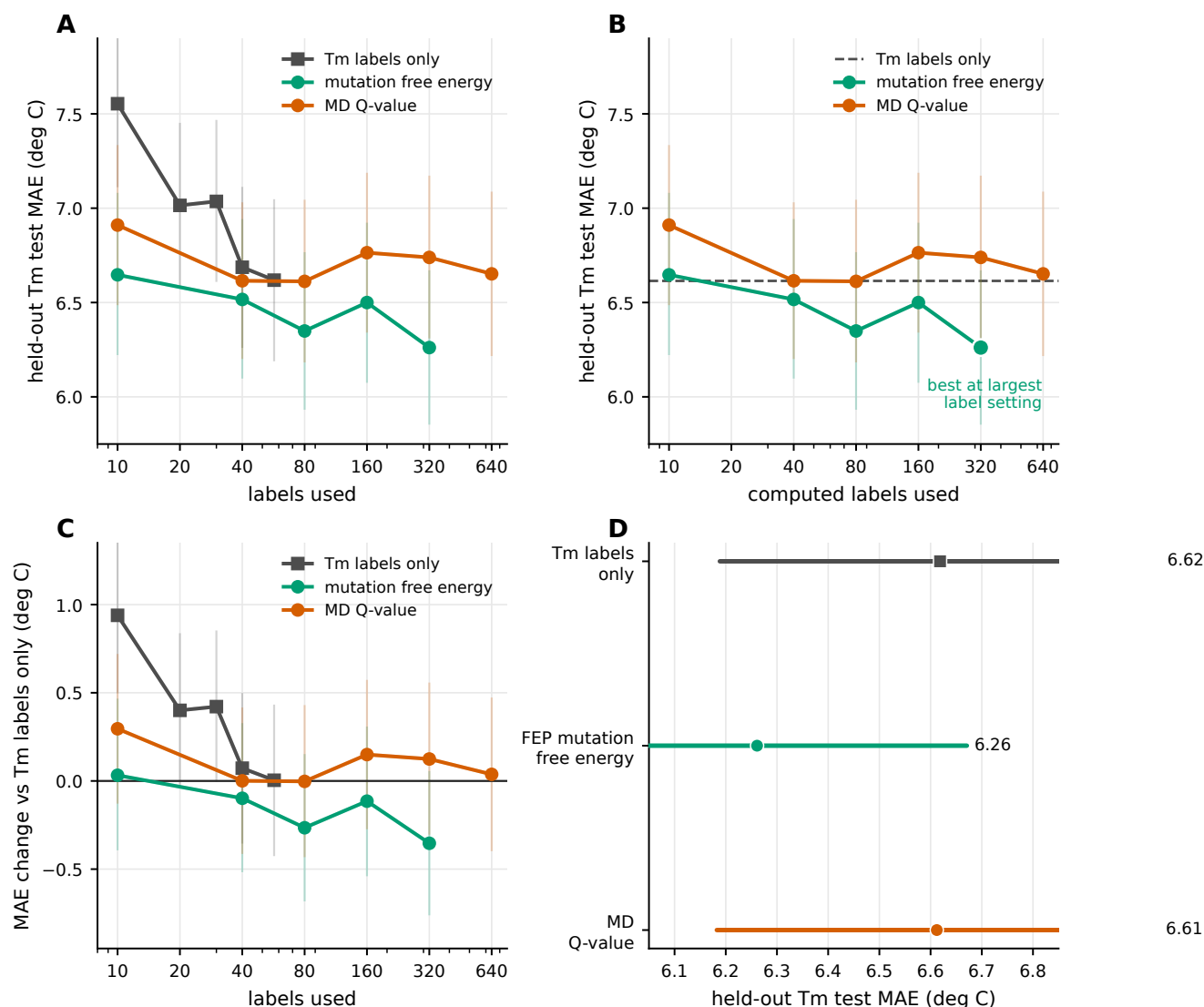
All sources are compared on the same test examples

This study set out to clarify how simulation-derived labels should be used when the experimental target is scarce and the simulated quantity is related to, but not the same as, that target. In nanobody Tm prediction, FEP mutation free-energy labels gave a modest but consistent improvement in held-out Tm prediction, while an MD-derived Q-value did not, and a larger ESM2 encoder did not reproduce the FEP gain either. The picture is selective: simulation data help when the source label encodes physical information that transfers to the experimental phenotype under target-centered validation.

The label that helped did not measure Tm. FEP estimates how a mutation changes free energy, whereas Tm is the temperature at which the folded and unfolded states balance under a given assay. These are not interchangeable quantities. Yet they describe the same stability landscape from different angles: sparse Tm measurements pin absolute stability for

the sequences we observe, and mutation free energies supply local directions in sequence space. The FEP result suggests that a source label need not be a direct surrogate for the target, as long as it describes a physically relevant perturbation of the target property.

Three controls sharpen this reading. First, the FEP label-count sweep was consistent with a benefit at the largest setting but was not smooth enough to claim a biomolecular scaling law; the firmer conclusion is source dependence. This both echoes and complicates the materials-informatics case, where the transferred error scales with the size of a related computational database (3): in our protein setting, the useful labels were not extra Tm measurements but mutation free energies. Second, model size was not the lever. The 35M and 650M ESM2 encoders did not beat the 8M encoder in either the Tm-only or FEP settings, and the best model paired the 8M encoder with FEP labels, in line with the broader finding that fine-tuning rather than raw capacity drives PLM-based protein-property prediction (16). Third, the frozen-encoder

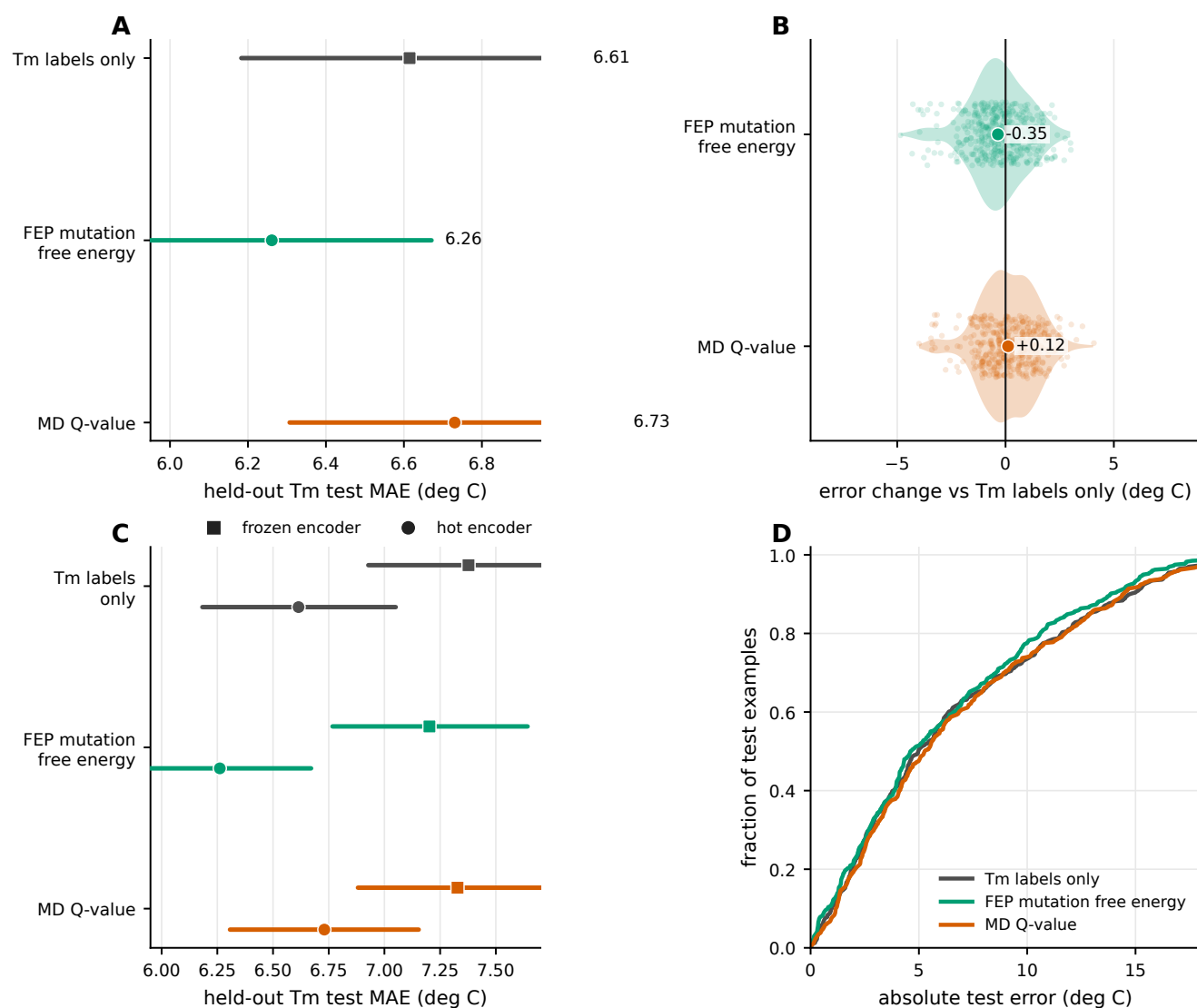


experiment locates the signal: FEP labels helped even with a fixed representation, but the best FEP model required updating the encoder. The labels work best when allowed to reshape the representation toward stability-relevant directions while still being selected on experimental Tm, which favors controlled encoder adaptation over either frozen feature extraction or unconstrained source-task optimization.

The MD Q-value result calls for the same specificity. It is not evidence that MD-derived descriptors are generally unhelpful; this particular high-temperature native-contact summary simply did not transfer under our protocol. By compressing each trajectory to a single native-contact persistence statistic, it may miss the mutation-specific energetics that govern how sequence changes shift stability. A simulation campaign aimed at target transfer should choose descriptors for their expected physical relevance to the endpoint, not for how many labels they yield.

The Rosetta-scored proposal-set result is best read as a feasibility test for design workflows. Rosetta scores on ESM2-proposed variants improved Tm prediction over Tm-only learning, but the direct comparison with random variants stayed within noise, so we do not claim that the present generator has already optimized nanobody thermostability. The narrower, solid point is that language-model proposals can be annotated by a physics-based score and used as supervised source data for a Tm predictor.

That supervised-scoring step is the only part of a design loop we tested. A full loop would let a generator propose variants, physics-based calculations annotate them, the Tm predictor update its estimate of experimental stability, and that estimate drive the next round of proposals—connecting this transfer-learning result to active or reinforcement learning. Those closed-loop steps, and experimental validation of newly proposed high-Tm nanobodies, are beyond the present scope.

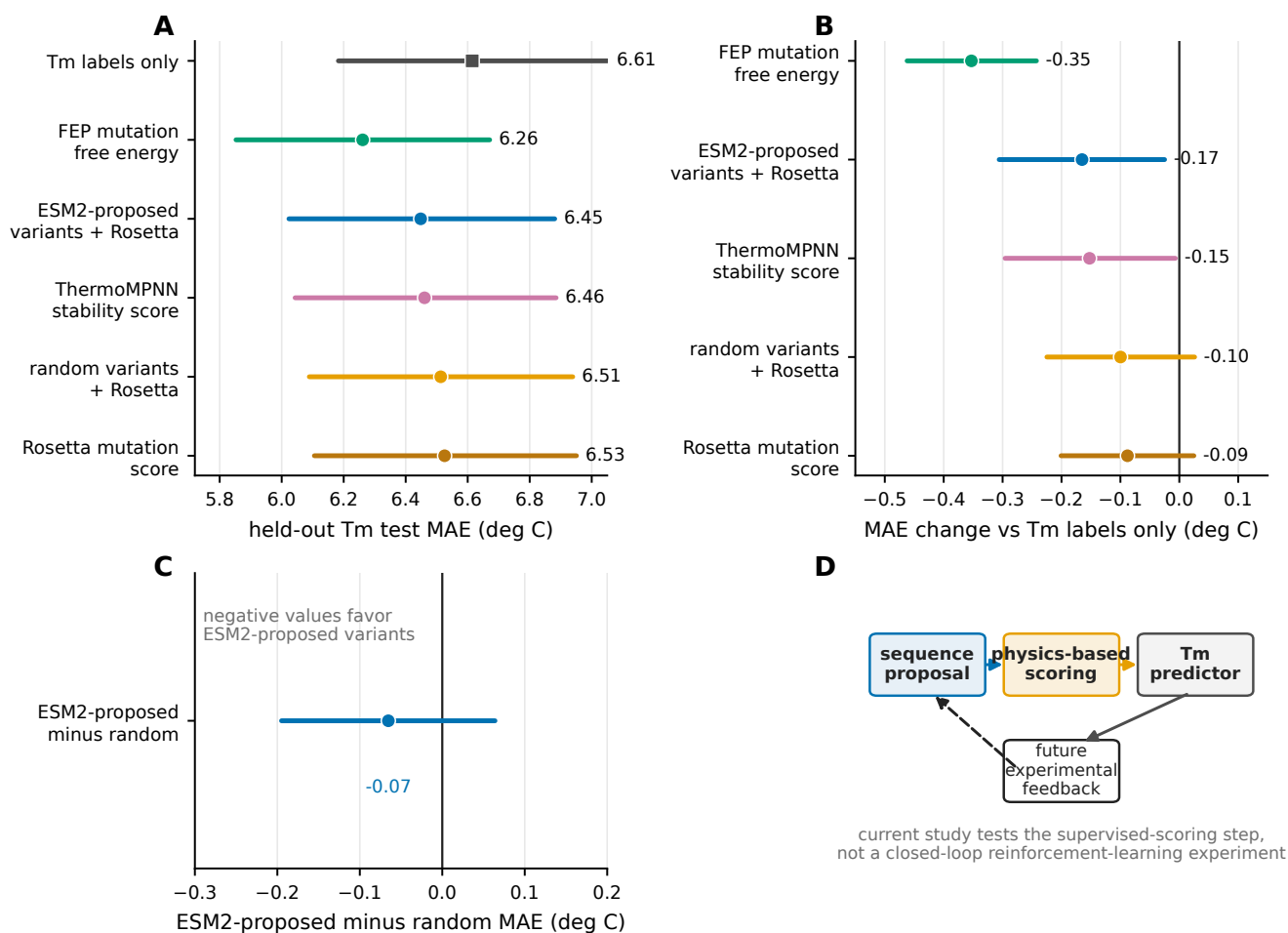


Several limitations bound the interpretation. The experimental Tm training set is small, so the conclusions should be retested on other nanobody collections and protein families. The source-label comparison covers only the labels available here; other free-energy protocols, stability predictors, or trajectory descriptors may transfer differently. The FEP curve supports a beneficial trend at the largest setting but does not establish a clean scaling exponent. No new experimental measurements were made to validate high-stability candidates. And because raw trajectories may be too large to deposit in full, reproducibility will rely on processed data, scripts, and a clearly reported simulation protocol.

For low-data experimental protein-property prediction, simulation labels are best treated as design choices rather than generic data augmentation. The question is not whether simulation can produce many labels, but whether those labels en-

code information that improves the target task under target-centered validation. Here, mutation free energies did that for nanobody Tm. The general recipe follows: use experimental data to define the target phenotype, use simulation to annotate physically meaningful perturbations around it, and judge the combined model only on held-out experimental data. That is a practical route from simulation-assisted prediction toward closed-loop protein engineering.

Taihei Murakami is an employee of Epsilon Molecular Engineering, Inc. Yasuhiro Matsunaga is a scientific advisor of Epsilon Molecular Engineering, Inc.



△

Processed data required to reproduce the main and supplementary figures will be deposited in Zenodo at . Raw simulation trajectories are large and will be made available at .

Analysis, training, and figure-generation code will be available on GitHub at . An archival release will be deposited in Zenodo at .

AI-assisted tools were used to support language editing, code development, and figure preparation. The authors reviewed and take responsibility for all manuscript content, analyses, and conclusions.

10.1021/acscentsci.9b00804
 10.1021/acs.macromol.2c02600
 10.1038/s41524-025-01606-5
 10.1146/annurev-biochem-063011-092449
 10.1186/s12951-024-02900-y
 10.1002/pro.3546
 10.1073/pnas.2016239118

10.1126/science.ade2574
 10.7554/eLife.82593
 10.1016/j.csbj.2023.11.006
 10.1038/s41598-024-70101-6
 10.1038/s41586-023-06328-6
 10.1371/journal.pcbi.1012248
 10.1093/database/baad021

10.1038/s41467-024-51844-2
 10.1093/nar/gkt1043
 10.1093/nar/gkh381
 K_a 10.1021/ct100578z
 10.1021/acs.jcim.3c01153
 10.1021/acs.jctc.9b00591
 10.1021/jz501780a
 10.1371/journal.pcbi.1005659
 10.1016/j.bpj.2015.08.015
 10.1073/pnas.1311599110
 10.1063/5.0014475
 10.1016/0263-7855(96)00018-5
 10.1021/acs.jcim.6b00162
 10.1038/s41586-024-07487-w
 $\phi\psi\chi_1\chi_2$ 10.1021/ct300400x
 10.1063/1.445869
 10.1016/0021-9991(77)90098-5
 10.1063/1.470117
 10.1016/0021-9991(76)90078-4
 10.1002/prot.22921
 10.1021/acs.jctc.7b00125
 10.1073/pnas.2314853121
 10.1109/CVPR.2018.00781

All target-task comparisons use the same NbBench thermo-seq endpoint (15). The processed tables hold an amino-acid sequence field and a melting-temperature label in degrees Celsius. The split sizes are 57 training, 114 validation, and 396 held-out test sequences. As described in Methods, we deliberately assigned the smallest NbBench partition to training and the largest to the test set so that the model sits in the low-data regime the study targets.

Split definitions. The splits are fixed throughout. The 57 training examples fit network parameters, the validation split drives checkpoint selection, early stopping, and choice among candidate settings, and the test split stays untouched until those choices are fixed. The same rule applies to Tm-only models and to models trained with computational source labels.

Target-label scaling. Before optimization, the training workflow maps the experimental Tm values to a scaled regression target. The scaler is fit only on the 57 training labels, using a robust scaling step followed by min–max scaling to [0,1]. Validation labels are transformed with this training-fitted scaler during model selection. Test predictions are averaged across seeds in scaled-label space and inverse-transformed to degrees Celsius before errors are computed, so the validation and test label distributions never influence the fitted transform.

Experimental-label scaling reference. This reference uses the same training split but subsamples it to the requested number of labels with the run seed; the validation and test splits are never subsampled. It serves only as a positive control for the pipeline: adding true target labels should lower the Tm prediction error.

Computational labels are loaded as source tasks rather than appended to the Tm data as extra measurements. Table 1 lists the processed target and source files used in the reported comparisons. Mutation-effect tables carry sequence fields and [0,1]-scaled source-label fields. The MD-derived Q-value table uses the same processed format so that it can enter the same source-task loader; there, the scaled label is a min–max scaled Q-value rather than a mutation free energy.

Data type	Role	Processed rows	Use in reported comparisons
Experimental Tm	Target train	57	Model fitting
Experimental Tm	Target validation	114	Checkpoint and setting selection
Experimental Tm	Target test	396	Final held-out evaluation
FEP mutation free energy	Source tasks	435 + 409	Main source task
Rosetta mutation score	Source tasks	435 + 409	Source comparison
ThermoMPNN stability score	Source tasks	435 + 409	Source comparison
Random variants scored by Rosetta	Source tasks	1000 + 1000	Source comparison
ESM2-proposed variants scored by Rosetta	Source tasks	1000 + 1000	Source comparison
MD-derived Q-value	Source task	1143	Source comparison and scaling control

Source-task assignment. The loader assigns each row to a target or source task. Experimental Tm rows use the Tm head. In the main mutation-effect model, the two template-specific tables use separate source heads, and the MD-derived Q-value table uses a single source head. This assignment fixes which regression head an example uses and which task-specific loss term it contributes to.

Sequence-overlap check. We checked exact sequence overlap between the processed source tables and the fixed NbBench splits. The FEP/Rosetta mutation-effect tables, the random variants scored by Rosetta, and the ESM2-proposed variants scored by Rosetta had no exact match to any NbBench training, validation, or test sequence. The ThermoMPNN tables used the same template-specific variant design as the mutation-effect sources. The SAbDab-derived MD Q-value table contained exact matches to 2 NbBench training, 4 validation, and 8 test sequences. We disclose this because it bears on interpretation of the MD control; it cannot explain the positive FEP result, and the MD Q-value source did not improve held-out Tm prediction.

Source-label sampling. For mutation-effect experiments with requested count n , the loader samples up to n rows independently from each of the two template tables using the run seed; the final FEP setting with $n = 320$ therefore supplies 320 sampled 1MEL rows and 320 sampled 4IDL rows before the source-task split. For MD-derived Q-value experiments, the loader samples n rows from the single Q-value table. Source rows are split 80/20 into source-training and source-validation rows with the same seed. The production training call then filters the validation data passed to the trainer so that checkpoint selection and early stopping depend only on the experimental Tm validation examples.

The MD source-label panel was built from SAbDab, the Structural Antibody Database (17). The archived SAbDab nanobody summary table holds 2422 rows with PDB identifier, heavy- and light-chain assignments, model index, antigen annotation,

experimental method, resolution, species, and other fields. The manifest workflow matches this summary against the archived IMGT-renumbered PDB files.

Structure-selection rule. The rule is deterministic. For each PDB identifier with both a SAbDab summary row and an IMGT-renumbered PDB file, the workflow takes the first heavy-chain row in the SAbDab table and accepts the chain only if that chain identifier is present in the IMGT PDB file. The output manifest holds one entry per PDB identifier, with fields for the manifest row number, PDB identifier, selected heavy chain, SAbDab TSV row, IMGT PDB path, and MD run directory. This initial one-per-PDB manifest contained 1177 structures.

400 K simulation panel. The 400 K campaign reused the systems prepared in the 300 K campaign and built a 1146-entry 400 K manifest, consisting of 798 X-ray, 345 electron-microscopy, and 3 NMR structures according to the SAbDab method field. None of the selected rows had an annotated light chain, consistent with a single-domain antibody panel. After trajectory and Q-value processing, 1143 unique PDB identifiers yielded usable source-label rows.

System preparation. Each structure was prepared as a nanobody-only system. The workflow takes the IMGT-renumbered PDB as the structure source, extracts the selected heavy chain, keeps only protein atoms, and leaves the termini uncapped, at pH 7.4. Chains are cleaned with PDBFixer, and pH-aware protonation is assigned through pdb2pqr/PROPKA when available (18, 19), with a fallback to AmberTools-based hydrogen assignment and Amber naming. The prepared chain is solvated in explicit OPC water with 15 Å padding and 0.15 M NaCl, and Amber topology and coordinate files are built with ff19SB and OPC (20–22).

300 K equilibration branch. The base 300 K branch uses OpenMM on CUDA devices (23): 1 ns of NVT followed by 1 ns of NPT at 300 K and 1 atm, with a 2-fs timestep and no hydrogen-mass repartitioning. Production at 300 K was generated for the broader campaign, but the source label used here comes from the 400 K branch.

400 K production branch. The first 400 K relaxation stage restarts from the 300 K equilibrated state and runs 1 ns of NVT at 400 K with C α restraints; the second repeats 1 ns of restrained 400 K NVT. Both relaxation stages use a 2-fs timestep and no hydrogen-mass repartitioning. Production runs 100 ns of restraint-free 400 K NVT, with hydrogen-mass repartitioning, 4 amu hydrogen mass, a 4-fs timestep, and coordinate/energy output every 100 ps.

Integrator and reporting settings. In OpenMM, explicit-solvent periodic systems use PME electrostatics with a 1.0-nm nonbonded cutoff and constraints on bonds to hydrogen. NPT stages use a Monte Carlo barostat; NVT stages omit it. The integrator is the Langevin middle integrator with 1 ps⁻¹ friction. The 100-ps reporting interval gives 1000 expected frames for a complete 100 ns production trajectory.

Trajectory input requirements. The processed MD table was generated from the archived 400 K production trajectories. A system was eligible only if it had a production trajectory, a prepared reference structure, and an Amber parameter file. The amino-acid sequence was read from the prepared structure using the first C α atom of each residue, and sequences shorter than 50 residues were excluded.

Native-contact definition. Q-values are computed with MDTraj (24). The topology is loaded from the Amber parameter file, and the native reference is frame 0 of production. The calculation uses protein backbone heavy atoms. A candidate native contact must span more than three residues, have a reference-frame distance below 0.45 nm, and include at least one atom from a hydrophilic residue (Asp, Glu, Gln, Asn, Arg, Lys, or His, including Amber protonation variants). For a native-contact pair (i, j) with reference distance r_{ij}^0 and instantaneous distance r_{ij} , the per-frame contribution is

$$q_{ij} = \frac{1}{1 + \exp\{\beta(r_{ij} - \lambda r_{ij}^0)\}},$$

with $\beta = 50 \text{ nm}^{-1}$ and $\lambda = 1.8$, following the smooth native-contact form of Best–Hummer-style Q-value analyses (25). The frame-level Q-value is the mean of q_{ij} over all selected native contacts.

Trajectory-level Q-value label. The trajectory-level label is the mean Q-value over the terminal portion of production. For trajectories with more than 300 frames, the implementation uses the larger of 300 frames and one third of the available frames; for shorter trajectories it uses all frames. For a complete 100 ns trajectory with 100-ps output, this is 333 terminal frames, roughly the final 33 ns. In the processed 400 K table, the number of frames used ranges from 66 to 333 (median 333), the number of native contacts from 28 to 728 (median 173), and the sequence length from 58 to 461 residues (median 122). Raw Q-values range from 0.0437 to 0.9994 and are min–max scaled to [0,1] for source-task training.

Sequences are tokenized with the ESM2 tokenizer for the selected encoder (8), using truncation and fixed-length padding to 160 residues. In the production protocol, the trainer receives all target and sampled source rows for training, but its evaluation data are filtered to the experimental Tm validation rows.

Hot-encoder input path. The hot-encoder setting keeps tokenized sequences in the training data and updates the ESM2 weights. The frozen-encoder setting runs ESM2 once per target and source sequence to precompute first-token embeddings, after which the optimizer updates only the regression trunk and task heads.

The default encoder is the 8M-parameter ESM2 model, and the model uses its first-token representation as the sequence-level embedding. The shared trunk has dimensions hidden-size $\rightarrow 256 \rightarrow 128 \rightarrow 32$, with ReLU activations and dropout after the first two hidden layers. The Tm head maps the 32-dimensional representation to one scaled scalar prediction.

Target and source heads. The Tm-only baseline is the encoder, the shared trunk, and the Tm head. Source-transfer models keep this target path and add source heads. The main mutation-effect model uses separate source heads for the two template-specific tasks. The code also implements source-head controls—a single shared mutation-effect head, a template-conditioned head, and a template-specific affine calibration on top of a shared head—to test whether the two template-specific sources need separate output calibration.

Architecture sensitivity analyses. Architecture controls include residual, latent, and mixture-style source-fusion variants, in which the source path produces a correction to the Tm prediction, a latent interaction between Tm and source features, or a gate between a target-only expert and a source-informed expert. The main reported FEP result uses the original shared-trunk architecture with separate template-specific source heads; the other variants are kept as sensitivity analyses, not as the main claim.

Each training example is a sequence, a scaled label, and a task identifier, and contributes loss only through the regression head for its task. The per-task loss is Huber loss with $\delta = 1.0$ on the scaled label. The default multi-task objective uses learned task-uncertainty weights (38): for task k with loss L_k and learned log-scale s_k , the contribution is

$$\frac{L_k}{2\exp(2s_k)} + s_k,$$

and the total loss sums the contributions from the tasks present in the minibatch. Fixed source-loss weights are also implemented and were included in the candidate searches for source classes where loss balance could shift validation performance.

Optimizer and schedule. The optimizer is AdamW. The default head/trunk learning rate is 3×10^{-4} , the default hot-encoder learning rate is 1×10^{-4} , weight decay is 0.04, dropout is 0.15, batch size is 16, warmup is 100 steps, and the schedule is cosine decay over at most 400 epochs. Checkpoints are evaluated and saved once per epoch, and mixed precision is used when CUDA is available. In the hot-encoder setting, the trainer creates separate optimizer parameter groups for ESM2 parameters and for the freshly initialized trunk/head parameters; in the frozen-encoder setting, ESM2 parameters are non-trainable and no encoder group is used.

Checkpoint selection. Within each run, the best checkpoint has the lowest mean squared error on the experimental Tm validation rows, and early stopping monitors the same metric with patience 15 and threshold 0. Source-task validation rows are not passed to the trainer in the production protocol. This detail is central to the comparison: source labels shape the learned representation through the training loss, but they do not decide which checkpoint is used for target-task prediction.

Candidate settings are selected on the experimental Tm validation split. The common grid includes the default setting; encoder learning rates of 3×10^{-5} , 1×10^{-4} , and 3×10^{-4} ; a lower head learning rate of 1×10^{-4} paired with encoder learning rate 3×10^{-5} ; and dropout rates of 0.05, 0.15, and 0.30. Fixed source-loss weights are added for source classes where the source-label scale may need explicit balancing. Tm-only candidates include shared, residual, and latent architecture variants; mutation-effect candidates use the shared-trunk family for the main comparison, with a separate source-head design comparison; and MD-derived candidates include fixed-weight settings because Q-value labels differ in scale and interpretation from mutation free energies.

Seeded candidate evaluation. Each candidate is first evaluated as a three-seed ensemble on the Tm validation split, and the selected setting for each source class is then evaluated on the held-out test split as a ten-seed ensemble. Final comparisons use seeds 1–10. The run seed controls Python, NumPy, PyTorch, and CUDA randomness when CUDA is available, as well as target-training subsampling in the target-label scaling experiment, source-row sampling, and source train/validation splitting. The label-count curves use fixed source-specific training recipes while the number of source labels is varied.

The primary metric is MAE in degrees Celsius on the 396 held-out test examples. For each condition, the ten seeded models produce scaled Tm predictions for every test sequence; these are averaged in scaled-label space and inverse-transformed

to degrees Celsius. Root mean squared error, coefficient of determination, and Spearman and Pearson correlations are also computed, but the figures emphasize MAE because it reads directly as an average temperature error.

Bootstrap intervals. Single-condition confidence intervals come from nonparametric bootstrap resampling over test examples. The absolute-error vector from the ten-seed ensemble is resampled with replacement 10,000 times using a fixed bootstrap seed, and the reported 90% interval is the 5th–95th percentile range of the bootstrapped MAE values. Paired comparisons use the same bootstrap indices for the candidate and reference absolute-error vectors. The reported ΔMAE is candidate minus reference, so negative values mean lower error than the reference.

Stored run summaries. Each training and evaluation call writes a structured summary recording the command-line arguments, relevant environment variables, resolved model settings, per-label-count metrics, bootstrap intervals, per-example absolute errors, and paired bootstrap summaries. Compact tables derived from these summaries feed the main source comparison, the frozen-encoder controls, the source-head controls, and the source-combination controls.

Target-label scaling reference. The target-label scaling curve is a positive control on the pipeline. It tests whether the same model family and evaluation code recover the expected gain when more true experimental Tm labels are available. The FEP source-label curve tests something different—whether more labels from a physically related but distinct computed quantity improve the same held-out Tm endpoint—while the MD Q-value curve tests whether simply adding more simulation-derived labels suffices when the source label measures a more indirect structural response. The decisive comparison is therefore not “simulation versus no simulation,” but whether the source label carries physical information that transfers to the target property under target-centered model selection.

The supplementary figures are generated from tabular source files derived from the same run summaries as the main figures. The processed source tables, rendered supplementary figures, and figure-generation code will be distributed with the public analysis archive.

The manuscript figures are regenerated from compact result summaries and source tables rather than from scratch run directories. The mapping below is the first place to check when auditing or extending a figure.

Display item	Inputs	Script	Primary comparison
Fig. 1	Conceptual protocol	Main-figure script	Target-centered transfer-learning design
Fig. 2	Main scaling summaries	Main-figure script	Label-count scaling for Tm labels, FEP labels, and MD Q-value labels
Fig. 3	Final hot- and frozen-encoder source-screen summaries	Main-figure script	All-atom FEP and MD labels versus training only on experimental Tm labels
Fig. 4	Final source-screen summary	Main-figure script	Rosetta, ThermoMPNN, and Rosetta-scored proposal sets
Supplementary Figs. 1–5	Supplementary manifest and generated TSV tables	Supplementary-figure script	Data-set composition, candidate-setting searches, model controls, scaling controls, and MD-feature controls

Generated supplementary tables. The final source-screen settings used in Figs. 3 and 4 and the frozen-encoder controls are recorded in generated supplementary tables, which list the selected architecture, validation-selected setting, source-label count, validation MAE, held-out test MAE, bootstrap interval, and run-summary reference for each condition. A panel-level mapping table for all supplementary figures is included in the public analysis archive.

